

SVKM'S NMIMS
School of Technology Management & Engineering
Department of Computer Engineering

AI

Artificial Intelligence

Project Report

Autonomous Embedded Firmware Porting and HAL Migration Orchestrator (HAL-Shift)

Submitted By:

Anupam Kanoongo
70022400311
Roll No: F044
B.Tech CE Section A

Submitted To:

Dr. Vikas Khare
Associate Professor
Associate Dean - STME

Architecture

1. System Overview

HAL-Shift is an enterprise-grade, Human-in-the-Loop (HITL) Multi-Agent System (MAS) designed to automate the porting of embedded C/C++ firmware between different microcontrollers (e.g., STM32 to ESP32). By combining Abstract Syntax Tree (AST) parsing, LLM-based logic mapping, strict approval gates, and headless virtual emulation, the system reduces migration times from months to days while ensuring total compliance and code safety.

2. Infrastructure & Container Orchestration

To ensure stateless execution and isolate dependencies (like specific cross-compilers or emulator versions), the system is deployed using container clusters.

- **Container Engine:** Docker / Kubernetes. Each agent operates in an isolated container instance.
- **Workflow Orchestrator:** An orchestrator (such as Apache Airflow, Temporal, or LangGraph) manages the sequential execution of agents, passes context/payloads between containers, and holds the system state while waiting for asynchronous human approvals.
- **Audit Database:** A read-only, immutable database (e.g., PostgreSQL) for compliance logging.

3. The Multi-Agent Pipeline

The pipeline operates sequentially, with clear boundaries of responsibility for each agent.

Phase 1: Context & Ground Truth

1. **System Trigger:** The Hardware Engineering Team inputs the legacy MCU and the target MCU details via the UI.
2. **Document Retrieval Agent:** Autonomously crawls, downloads, and validates the most up-to-date datasheets, SDK manuals, and reference documents for the target MCU.
3. **Hardware Analysis Agent:** Consumes the retrieved documentation. It synthesizes a highly structured **Hardware Markdown** detailing clock tree changes, specific pinout mappings, and peripheral limits.
4. **Software Analysis Agent:** Consumes both the legacy C/C++ codebase (via Clang AST parsing) AND the Hardware Markdown. It generates a **Software Markdown** outlining the exact driver refactoring, struct replacements, and logic shifts needed.

Phase 2: The HITL Verification Gate (The "AND" Gate)

The pipeline pauses. The Hardware Markdown is routed to the Hardware Head, and the Software Markdown is routed to the Software Head for asynchronous review. The system orchestrator waits for (HW_Approval == True) AND (SW_Approval == True).

Phase 3: Execution & Verification

5. **Implementation Agent:** Locked strictly behind the AND gate. Once triggered, it executes the code changes dictated by the approved Software Markdown and pushes a Git branch.
6. **Virtual Verification Agent:** Compiles the new branch and runs it headlessly in a virtual hardware emulator (Renode/QEMU) to verify functional equivalence without physical hardware.

4. Strict State Management & Feedback Loops

The core safety mechanism of HAL-Shift is its robust handling of iterative feedback during Phase 2. Reviewers have three explicit options:

- **Approve:** Sets the department's boolean flag to True.
- **Minor Change:** The reviewer provides specific text feedback (e.g., "Change I2C pins from A4/A5 to B6/B7").
  - *The Mutual Reset Rule:* If Team A requests a Minor Change, the orchestrator updates the context and triggers the agent to regenerate the document. Crucially, if Team B had already approved their document, **Team B's approval flag is forcibly reset to False**. Both teams must review and re-approve the "fresh docs" to prevent hidden dependency conflicts.
- **Major Change (Hard Reset):** The reviewer identifies a fundamental flaw (e.g., the agent retrieved the wrong datasheet). The orchestrator aborts the current container pipeline, wipes the state memory, and restarts the entire process from the Document Retrieval Agent to prevent cascading hallucinations.

5. UI/UX & Cognitive Load Management

To prevent alert fatigue and ensure thorough reviews, the UI adapts based on the iteration cycle:

- **Iteration 0 (Baseline Read):** For the very first review pass, the UI enforces a full, raw read of the generated Markdowns. No summaries or shortcuts are provided, forcing both teams to establish a shared, undisputed ground truth.
- **Iteration 1+ (Minor Change Loops):** When "fresh docs" are sent back due to a Minor Change loop, the UI provides a toggle:
  - **Visual Diff:** A Git-style view highlighting exactly what text was added (green) or removed (red) since the last version.
  - **AI Summary:** A generated, high-level summary of the cascading impacts (e.g., *"Agent Note: Hardware team updated I2C pins; Software logic updated to reflect new port mapping below."*)
  - **Traditional Full Review:** A fallback mode that presents the full, unsummarized documentation, ensuring reviewers can manually verify every detail without AI-assisted filtering or diffing.

6. Enterprise Governance & Compliance

Because embedded systems often reside in regulated spaces (medical, automotive), HAL-Shift enforces strict accountability:

- **Role-Based Access Control (RBAC):** Approval routing is strictly limited to users designated as "Engineering Heads" (Tech Leads, Architects). Junior developers cannot unlock the implementation gate.
- **Immutable Audit Trail:** Upon final approval (the opening of the AND gate), the system writes a permanent record to the Audit Database containing:
  1. The SSO identity of the approvers.
  2. Exact timestamp of the mutual approval.
  3. The SHA-256 Git Hashes of the exact Hardware and Software Markdowns that were approved.
  4. A complete log of all Minor/Major changes requested during the pipeline execution.

About the DataSet

HAL-Shift Synthetic Migration Dataset represents completed firmware-migration projects processed through the HAL-Shift Human-in-the-Loop Multi-Agent System, which automates porting embedded C/C++ firmware between microcontrollers. Each row is one completed migration, capturing metrics from MCU-pair selection through final sign-off.

The dataset has **50,000 rows** and **13 columns** in a clean CSV with no missing values: an identifier, four categorical context variables, and continuous metrics covering project scale, review activity, and outcomes. The numeric columns are driven by two hidden latent factors: Project Complexity and Review Friction, so the data behaves realistically under descriptive statistics, factor analysis, and regression.

Attribute	Description
migration_id	Unique project identifier
legacy_mcu_family	Source microcontroller family (e.g. STM32F4, ATmega328)
target_mcu_family	Destination microcontroller family (e.g. ESP32, nRF52840)
industry_domain	Regulatory context: Automotive, Medical, Industrial, or Consumer
emulator_type	Virtual hardware emulator used for verification (Renode or QEMU)
codebase_size_loc	Size of the legacy codebase, in lines of code
num_peripherals	Number of distinct hardware peripherals touched by the port
iteration_count	Number of Human-in-the-Loop review cycles before mutual approval
num_minor_changes	Total "Minor Change" feedback requests raised during review
num_major_changes	Number of "Major Change" hard resets requiring a full pipeline restart
hw_review_time_hrs	Hours the Hardware Head spent reviewing the Hardware Markdown
sw_review_time_hrs	Hours the Software Head spent reviewing the Software Markdown
verification_pass_rate	Percentage of functional-equivalence tests passed in emulation
total_pipeline_duration_days	Total elapsed time from project trigger to final approval

PEAS Analysis

Top-Level System PEAS (HAL-Shift Pipeline Orchestrator)

This defines the central workflow engine managing the state, the AND-gate logic, and the UI delivery to the Engineering Heads.

- **Performance Measure:**
  - Strict adherence to the (**HW_Approval == True**) AND (**SW_Approval == True**) synchronization logic.
  - Successful capture of immutable audit trails (timestamp, approver identity, markdown Git hash) for 100% of pipeline executions.
  - Reduction of "alert fatigue" through accurate visual diff generation and AI summarization for iterative loops.
  - Safe abortion and state clearing upon a "**Major Change**" (Hard Reset) trigger.
- **Environment:**
  - Containerized infrastructure (e.g., Docker/Kubernetes clusters).
  - Web-based UI dashboard or integration with enterprise communication tools (Slack/Teams/Jira) for Head approvals.
  - Audit log database (read-only compliance storage).
  - Legacy embedded C/C++ repository.
- **Actuators:**
  - State Manager: Toggling the internal boolean approval flags based on user input and enforcing mutual resets.
  - UI Renderer: Generating the "**Raw View**" (Pass 0), the "**Visual Diff**", and the "**AI Summary**" for the Engineering Heads.
  - Audit Logger: Writing compliance data to the database upon approval.
  - Agent Trigger: Activating subsequent containers (e.g., waking the Implementation Agent only when the AND gate opens).
- **Sensors:**
  - User Input Webhooks: Receiving the exact choice (**Approve, Minor Change, Major Change**) and **typed feedback** from the Engineering Heads.
  - Container State Probes: Monitoring the success/failure status of the individual agent containers.

2. Agent-Specific PEAS Analyses

2.1 Document Retrieval Agent

- This agent acts as the foundation, ensuring the system bases its logic on reality, not hallucinations.
- **Performance Measure:**
  - 100% accuracy in fetching the correct manufacturer (e.g., STMicroelectronics, Espressif) datasheet for the exact target **MCU part number.**
  - Speed of retrieval and extraction of relevant register/pinout tables.
- **Environment:**
  - The public web, specific manufacturer **API endpoints**, and technical documentation databases.
- **Actuators:**
  - Web crawlers and PDF downloaders.
  - Document storage writers (saving the raw PDFs/HTML to the container volume).
- **Sensors:**
  - HTTP response readers.
  - Search engine query returns.

2.2 Hardware Analysis Agent

This agent consumes the raw documentation to create the physical ground truth.

- **Performance Measure:**
  - Accuracy of the extracted clock tree boundaries, pinout mappings, and peripheral limits.
  - Clarity and structure of the generated "Hardware Markdown."
  - Ability to accurately tweak specific markdown sections without losing surrounding context during a "Minor Change" loop.
- **Environment:**
  - The validated documents provided by the Retrieval Agent.
  - Text-based feedback injected from the Hardware Engineering Head (during loops).
- **Actuators:**
  - Markdown Generator: Writing the structured Hardware Markdown file.
- **Sensors:**
  - Document parsers (extracting tables and text from PDFs/HTML).
  - Feedback string readers.

2.3 Software Analysis Agent

This agent bridges the physical hardware reality with the legacy code logic.

- **Performance Measure:**
  - Accurate identification of all legacy HAL drivers that need swapping.
  - Perfect alignment with the facts stated in the Hardware Markdown (no contradictions).
  - Clear architectural mapping of necessary code changes (the "Software Markdown").
- **Environment:**
  - The legacy C/C++ codebase.
  - The verified "Hardware Markdown" (acts as strict contextual boundary).
  - Text-based feedback injected from the Software Engineering Head.
- **Actuators:**
  - Markdown Generator: Writing the structured Software Markdown file.
- **Sensors:**
  - Codebase indexers/AST parsers.
  - Text readers (consuming the Hardware Markdown).

2.4 Implementation Agent

The final executor, strictly locked behind the HITL approval gate.

- **Performance Measure:**
  - Syntactic correctness of the final generated C/C++ code.
  - 1:1 exact reflection of the instructions approved in the Software Markdown (no unapproved improvisations).
- **Environment:**
  - The legacy C/C++ codebase.
  - The mutually approved, final versions of both Markdowns.
- **Actuators:**
  - Code Generator: Rewriting and injecting the new C/C++ syntax.
  - Version Control Client: Creating a Git branch, committing the changes, and opening the final Pull Request.
- **Sensors:**
  - The Orchestrator's "GO" signal (the AND gate unlocking).
  - File system readers (to view the code it needs to rewrite).

Descriptive Statistics - Summary Tables Report

Dataset C:\Users\vboxuser\Documents\hal_shift.NCSS

Summary Table

Statistic	Variable	
	codebase_size_loc	**num_peripherals**
Count	50000	50000
Sum	4.004912E+08	300131
Mean	8009.824	6.00262
Standard Deviation	3447.195	2.43402
Standard Error	15.41632	0.01088527
Lower 95% CL Mean	7979.608	5.981285
Upper 95% CL Mean	8040.04	6.023955
Median	7998	6
Minimum	500	1
Maximum	23299	17
Range	22799	16
Variance	1.188315E+07	5.924452
Skewness	0.08268142	0.1067265
Kurtosis	2.786467	2.739552

Statistic	Variable	
	iteration_count	**num_minor_changes**
Count	50000	50000
Sum	131097	148020
Mean	2.62194	2.9604
Standard Deviation	1.723064	2.475365
Standard Error	0.007705777	0.01107017
Lower 95% CL Mean	2.606837	2.938703
Upper 95% CL Mean	2.637043	2.982097
Median	3	3
Minimum	0	0
Maximum	10	14
Range	10	14
Variance	2.96895	6.127434
Skewness	0.3169701	0.6207181
Kurtosis	2.613091	2.795523

Descriptive Statistics - Summary Tables Report

Dataset C:\Users\vboxuser\Documents\hal_shift.NCSS

Summary Table (Continued)

Statistic	num_major_changes	hw_review_time_hrs
Count	50000	50000
Sum	9628	265090.1
Mean	0.19256	5.301802
Standard Deviation	0.4772056	3.611949
Standard Error	0.002134128	0.01615313
Lower 95% CL Mean	0.1883772	5.270143
Upper 95% CL Mean	0.1967428	5.333462
Median	0	5
Minimum	0	0.5
Maximum	6	24.8
Range	6	24.3
Variance	0.2277252	13.04618
Skewness	2.820232	0.5097874
Kurtosis	12.55126	2.709044

Statistic	sw_review_time_hrs	verification_pass_rate
Count	50000	50000
Sum	323726.6	4740437
Mean	6.474532	94.80874
Standard Deviation	4.329576	4.849629
Standard Error	0.01936245	0.0216882
Lower 95% CL Mean	6.436582	94.76623
Upper 95% CL Mean	6.512482	94.85125
Median	6.2	95.7
Minimum	0.5	61.9
Maximum	26.6	100
Range	26.1	38.1
Variance	18.74523	23.51891
Skewness	0.4594913	-0.9176264
Kurtosis	2.671553	3.538723

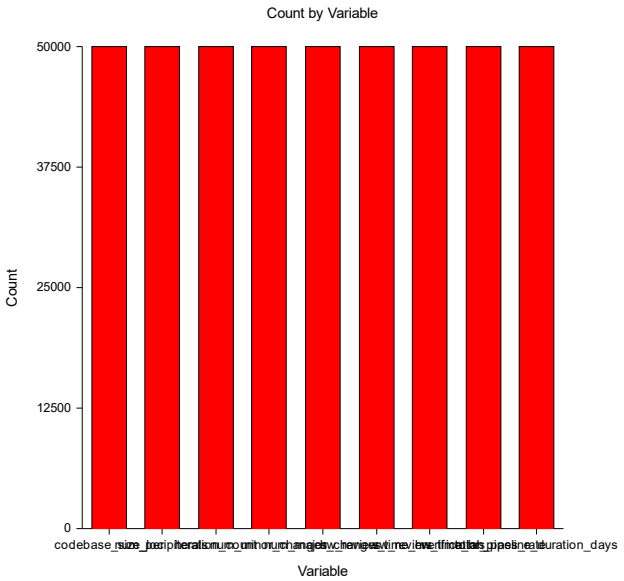
Descriptive Statistics - Summary Tables Report

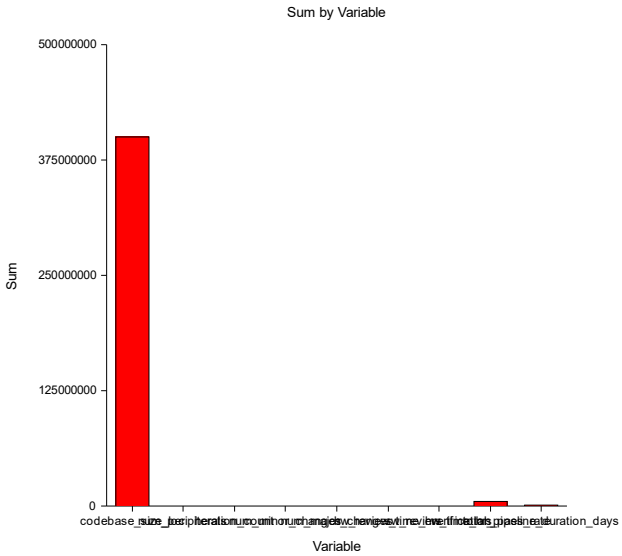
Dataset C:\Users\vboxuser\Documents\hal_shift.NCSS

Summary Table (Continued)

Statistic	Variable total_pipeline_duration_days
Count	50000
Sum	676666.3
Mean	13.53333
Standard Deviation	8.387993
Standard Error	0.03751224
Lower 95% CL Mean	13.4598
Upper 95% CL Mean	13.60685
Median	12.7
Minimum	1
Maximum	68.8
Range	67.8
Variance	70.35842
Skewness	0.6324941
Kurtosis	3.358196

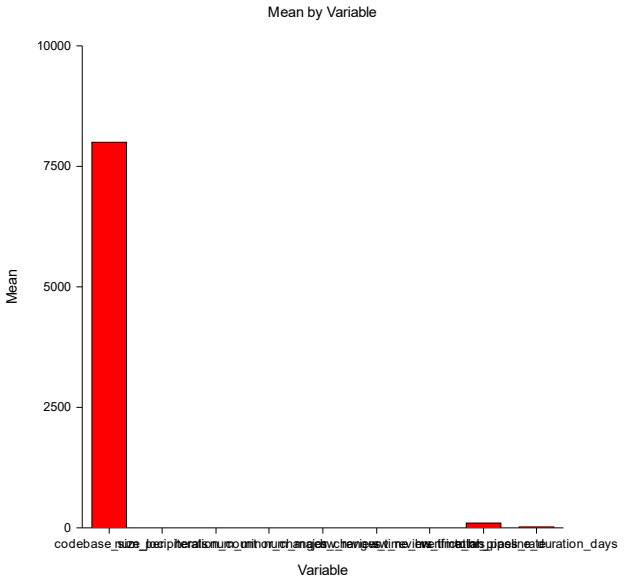
Plots of each Statistic

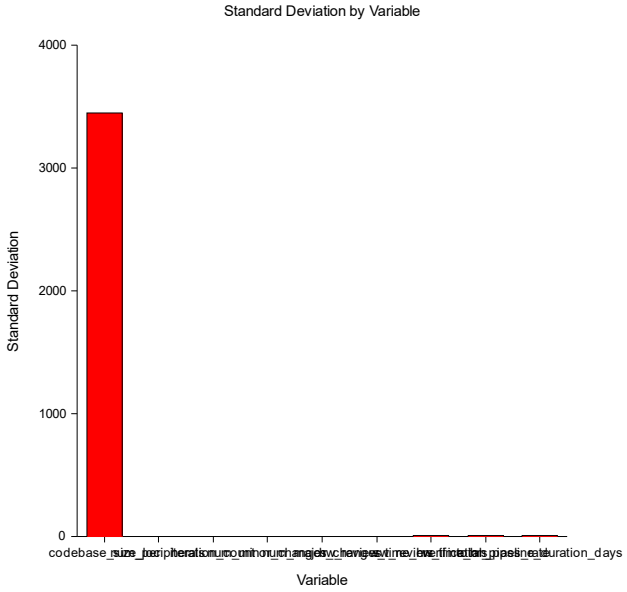


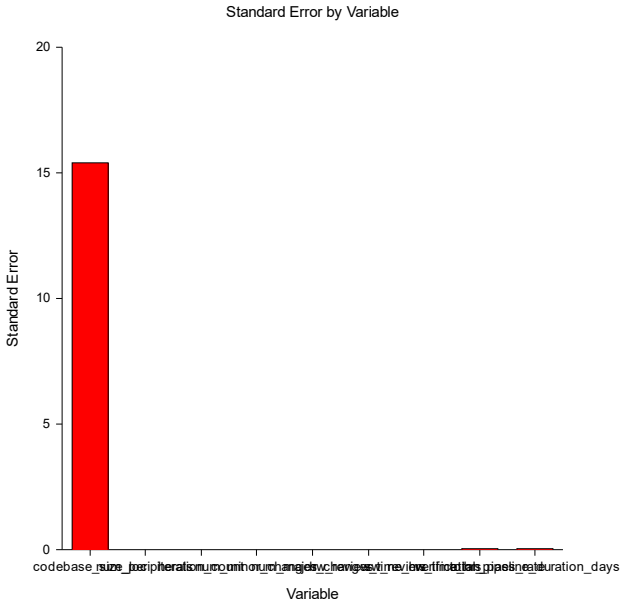


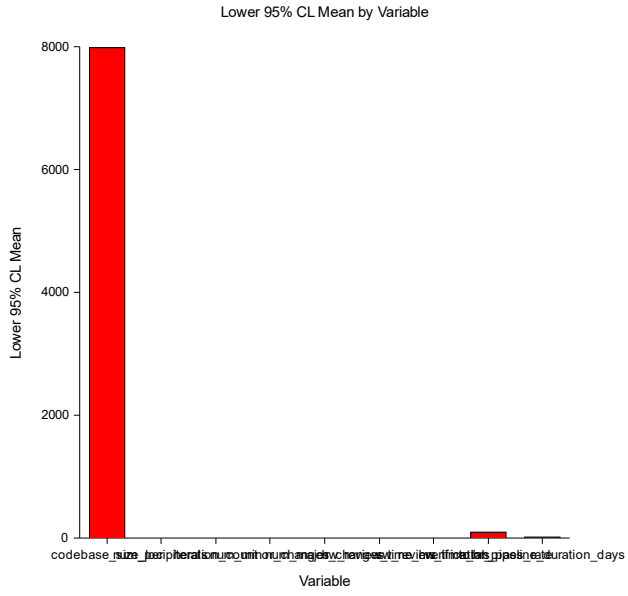
Descriptive Statistics - Summary Tables Report

Dataset C:\Users\vboxuser\Documents\hal_shift.NCSS



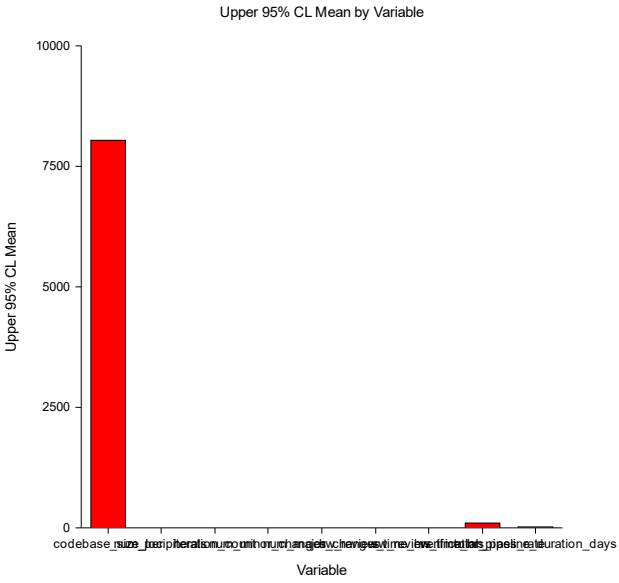


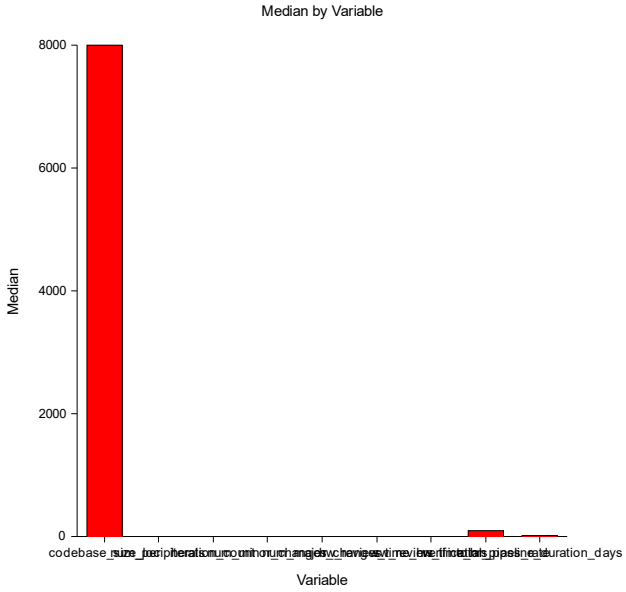


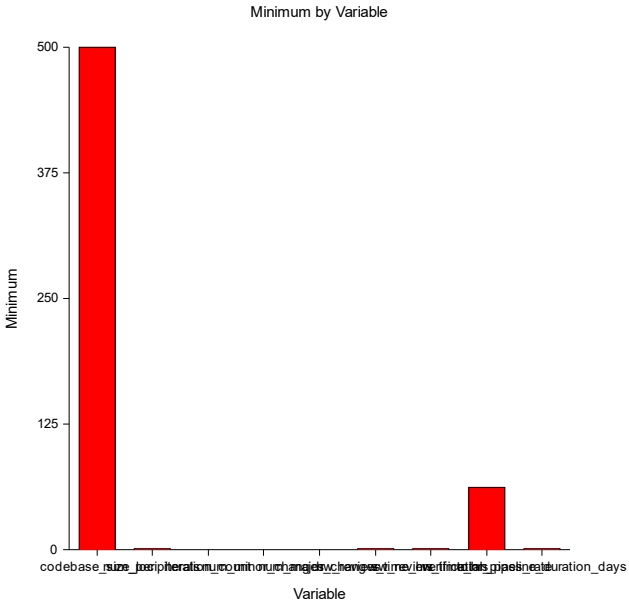


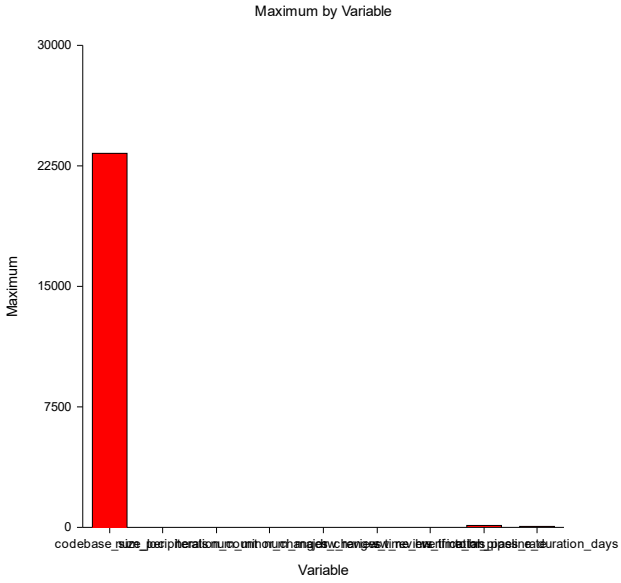
Descriptive Statistics - Summary Tables Report

Dataset C:\Users\vboxuser\Documents\hal_shift.NCSS



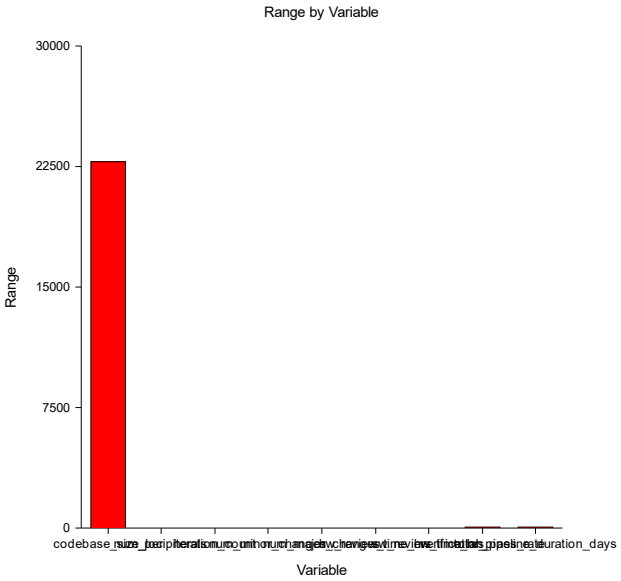


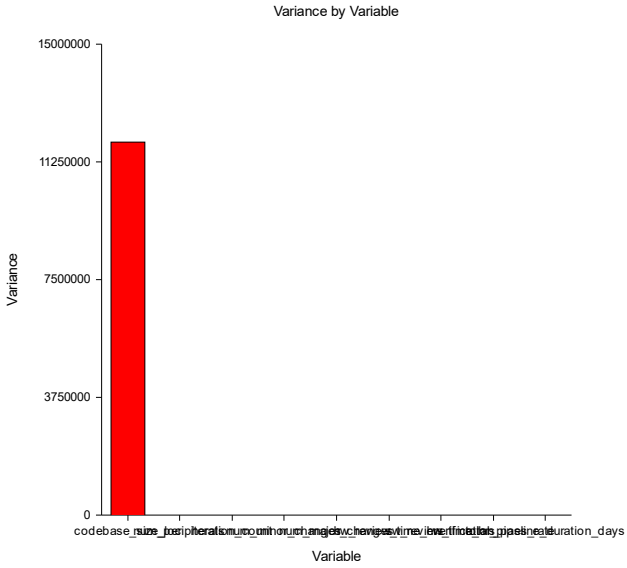


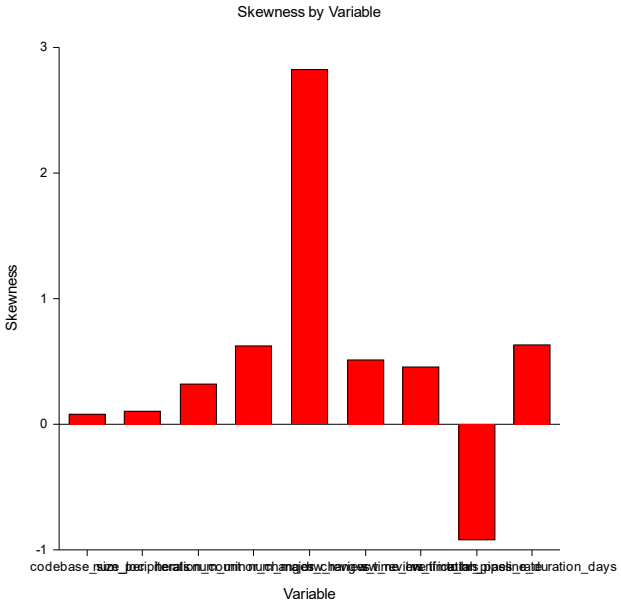


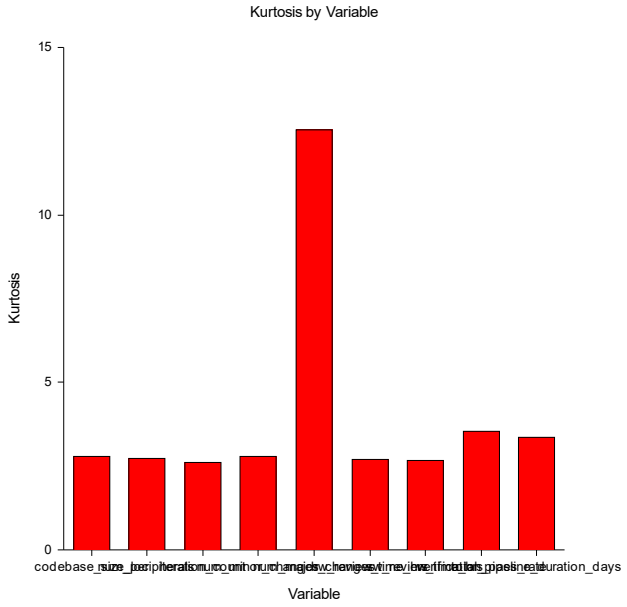
Descriptive Statistics - Summary Tables Report

Dataset C:\Users\vboxuser\Documents\hal_shift.NCSS









Frequency Table Report

Dataset C:\Users\vboxuser\Documents\hal_shift.NCSS

Frequency Distribution of legacy_mcu_family

legacy_mcu_family	Count	Cumulative Count	Percent	Cumulative Percent	Graph of Percent
ATmega328	8968	8968	17.94%	17.94%	|||||||
MSP430	6086	15054	12.17%	30.11%	||||
PIC32MX	7071	22125	14.14%	44.25%	|||||
STM32F1	10959	33084	21.92%	66.17%	||||||||
STM32F4	10966	44050	21.93%	88.10%	||||||||
STM32L0	5950	50000	11.90%	100.00%	||||

Frequency Distribution of target_mcu_family

target_mcu_family	Count	Cumulative Count	Percent	Cumulative Percent	Graph of Percent
ESP32	14976	14976	29.95%	29.95%	||||||||||
ESP32-S3	11957	26933	23.91%	53.87%	|||||||||
RP2040	7901	34834	15.80%	69.67%	||||||
STM32H7	5054	39888	10.11%	79.78%	||||
nRF52840	10112	50000	20.22%	100.00%	||||||||

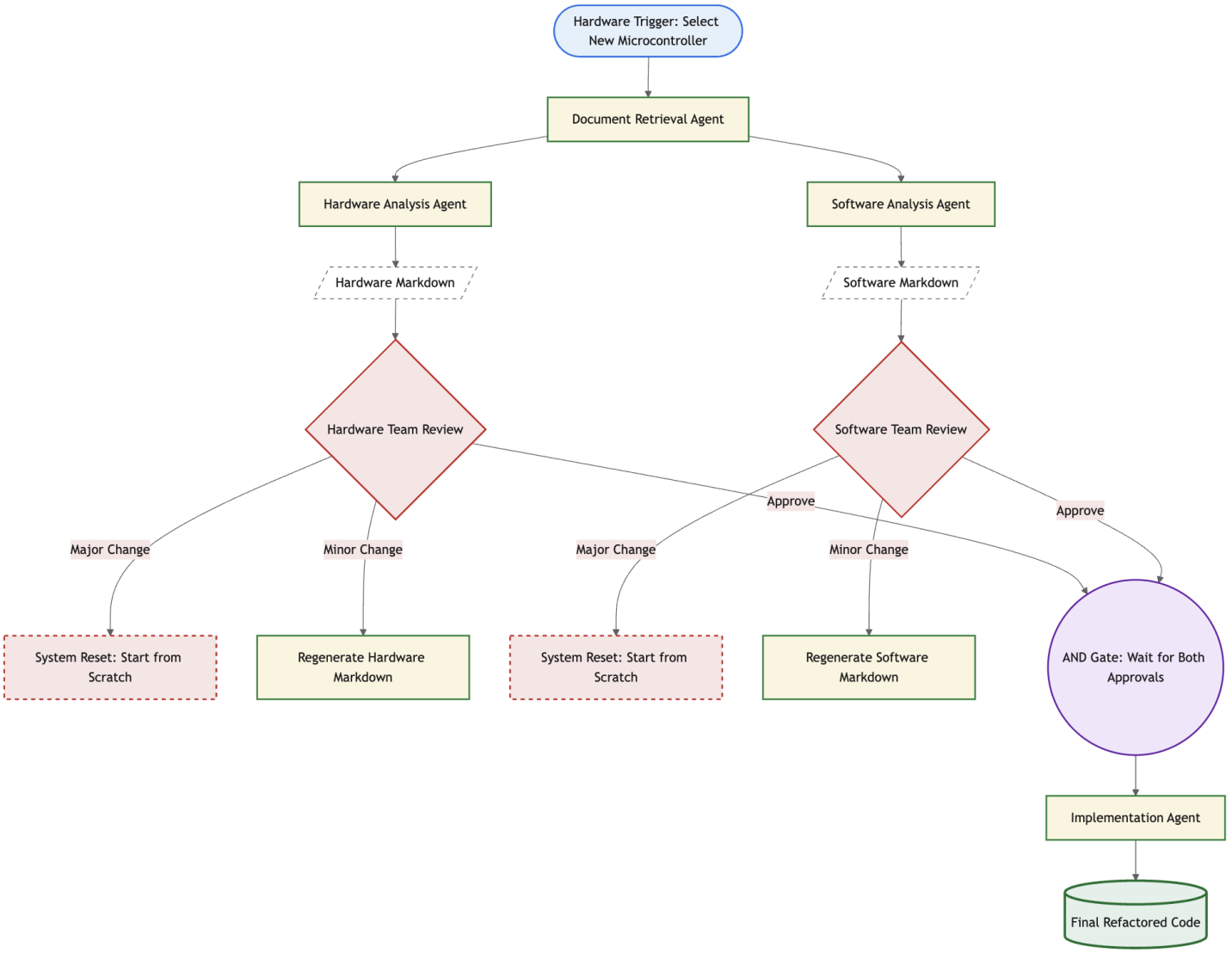
Frequency Distribution of industry_domain

industry_domain	Count	Cumulative Count	Percent	Cumulative Percent	Graph of Percent
Automotive	14089	14089	28.18%	28.18%	||||||||||
Consumer	12513	26602	25.03%	53.20%	|||||||||
Industrial	13397	39999	26.79%	80.00%	|||||||||
Medical	10001	50000	20.00%	100.00%	||||||||

Frequency Distribution of emulator_type

emulator_type	Count	Cumulative Count	Percent	Cumulative Percent	Graph of Percent
QEMU	22427	22427	44.85%	44.85%	||||||||||||||||
Renode	27573	50000	55.15%	100.00%	||||||||||||||||||||||

BFS / DFS Approach



The HAL-Shift process flow diagram illustrates a secure, deterministic orchestration pipeline that prioritizes human oversight and data integrity during microcontroller migrations. By initiating the workflow from a single hardware trigger, the system strictly divides responsibilities between parallel Hardware and Software Analysis Agents, generating isolated Markdown documents that establish a clear "ground truth." The core strength of this architecture lies in the Human-in-the-Loop (HITL) review phase, which enforces a mandatory "AND Gate" synchronization; the final Implementation Agent is completely locked until both the hardware and software engineering teams grant explicit approval. Furthermore, the inclusion of distinct "Minor Change" and "Major Change" (System Reset) branches ensures that iterative adjustments are handled gracefully, while catastrophic errors trigger a full system reset to prevent the compounding of LLM hallucinations or dependency conflicts.

1. Breadth-First Search (BFS)

BFS explores the diagram level-by-level, moving horizontally across parallel processes before moving down.

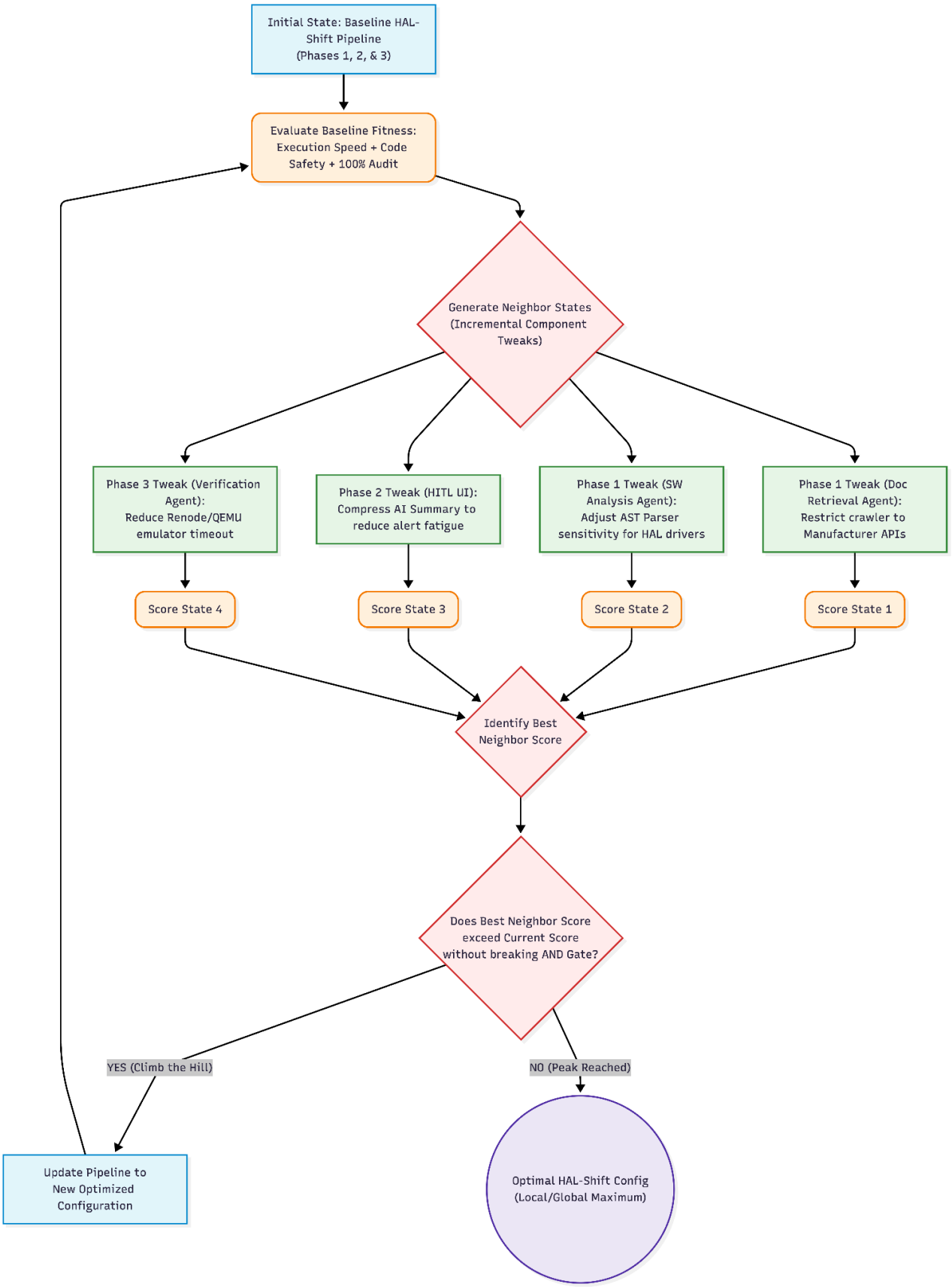
1. Hardware Trigger: Select New Microcontroller
2. Document Retrieval Agent
3. Hardware Analysis Agent
4. Software Analysis Agent
5. Hardware Markdown
6. Software Markdown
7. Hardware Team Review
8. Software Team Review
9. System Reset: Start from Scratch (Hardware Major Change)
10. Regenerate Hardware Markdown
11. System Reset: Start from Scratch (Software Major Change)
12. Regenerate Software Markdown
13. AND Gate: Wait for Both Approvals
14. Implementation Agent
15. Final Refactored Code

2. Logical Depth-First Search (DFS)

A Logical DFS dives as far down a single branch as possible. However, when it encounters a synchronization point (the AND Gate), it cannot logically proceed until all dependencies are met. It pauses that branch, backtracks to resolve the missing dependencies, and only resumes depth traversal when the gate unlocks.

1. Hardware Trigger: Select New Microcontroller
2. Document Retrieval Agent
3. Hardware Analysis Agent
4. Hardware Markdown
5. Hardware Team Review
6. System Reset: Start from Scratch (Hardware Major Change) (Backtracks)
7. Regenerate Hardware Markdown (Backtracks)
8. AND Gate: Wait for Both Approvals (Logical Block: Pauses depth traversal here, waiting on Software. Backtracks to Document Retrieval Agent)
9. Software Analysis Agent
10. Software Markdown
11. Software Team Review
12. System Reset: Start from Scratch (Software Major Change) (Backtracks)
13. Regenerate Software Markdown (Backtracks)
14. AND Gate: Wait for Both Approvals (Condition Met: Both approvals are now present. Depth traversal resumes)
15. Implementation Agent
16. Final Refactored Code

Hill Climbing Approach



This flowchart illustrates a Steepest-Ascent Hill Climbing algorithm specifically tailored to optimize the HAL-Shift multi-agent orchestration pipeline.

1. Initialization and Baseline Evaluation

The loop begins with the initial state: the baseline HAL-Shift pipeline operating across Phase 1 (Context & Ground Truth), Phase 2 (The HITL Verification Gate), and Phase 3 (Execution & Verification). The system evaluates this baseline's fitness using a composite score of execution speed, code safety, and the successful capture of immutable audit trails.

2. Generating Neighbor States

The algorithm explores adjacent configurations by applying isolated, incremental tweaks to four key components:

- **Document Retrieval Agent (Phase 1):** Restricting the crawler search space exclusively to manufacturer APIs to streamline datasheet fetching.
- **Software Analysis Agent (Phase 1):** Adjusting the AST parser's sensitivity to more efficiently identify legacy HAL drivers.
- **HITL UI (Phase 2):** Compressing AI summaries to reduce reviewer "alert fatigue" during iteration loops.
- **Virtual Verification Agent (Phase 3):** Incrementally reducing the Renode/QEMU emulator timeout to save compute time.

3. Scoring and The Constraint Check

Each tweaked state (State 1 through 4) is scored independently. The algorithm identifies the highest-scoring neighbor and submits it to a critical decision gate. For the system to adopt the new state, it must improve the overall fitness score **without** violating the pipeline's strict synchronization logic: waiting for (HW_Approval == True) AND (SW_Approval == True).

4. Climb or Terminate

- **YES (Climb the Hill):** If the highest-scoring neighbor improves efficiency while maintaining the "AND Gate" compliance, the pipeline configuration is updated, and the evaluation loop restarts.
- **NO (Peak Reached):** If no incremental tweak improves the score without breaking safety constraints, the algorithm halts, outputting the optimal HAL-Shift configuration as a local or global maximum.

Factor Analysis Report

Dataset C:\Users\vboxuser\Documents\hal_shift.NCSS

Descriptive Statistics Section

Variables	Count	Mean	Standard Deviation	Communality
codebase_size_loc	50000	8009.824	3447.195	0.745813
num_peripherals	50000	6.00262	2.43402	0.640574
iteration_count	50000	2.62194	1.723064	0.873446
num_minor_changes	50000	2.9604	2.475365	0.880072
num_major_changes	50000	0.19256	0.4772056	0.231248
hw_review_time_hrs	50000	5.301802	3.611949	0.706336
sw_review_time_hrs	50000	6.474532	4.329576	0.744681
verification_pass_rate	50000	94.80874	4.849629	0.766294

Correlation Section

	Variables				
Variables	**codebase_size_loc**	**num_peripherals**	**iteration_count**	**num_minor_changes**	**num_major_changes**
codebase_size_loc	1.000000	0.770011	0.275536	0.271930	0.337923
num_peripherals	0.770011	1.000000	0.251368	0.249380	0.310972
iteration_count	0.275536	0.251368	1.000000	0.902512	0.110502
num_minor_changes	0.271930	0.249380	0.902512	1.000000	0.110431
num_major_changes	0.337923	0.310972	0.110502	0.110431	1.000000
hw_review_time_hrs	0.451465	0.411310	0.735180	0.737602	0.176215
sw_review_time_hrs	0.520431	0.476163	0.728722	0.730849	0.210562
verification_pass_rate	-0.720083	-0.660494	-0.235753	-0.236011	-0.556587

Phi=0.511413 Log(Det|R|)=-6.084417 Bartlett Test=304193.45 DF=28 Prob=0.000000

	Variables		
Variables	**hw_review_time_hrs**	**sw_review_time_hrs**	**verification_pass_rate**
codebase_size_loc	0.451465	0.520431	-0.720083
num_peripherals	0.411310	0.476163	-0.660494
iteration_count	0.735180	0.728722	-0.235753
num_minor_changes	0.737602	0.730849	-0.236011
num_major_changes	0.176215	0.210562	-0.556587
hw_review_time_hrs	1.000000	0.759949	-0.387968
sw_review_time_hrs	0.759949	1.000000	-0.453485
verification_pass_rate	-0.387968	-0.453485	1.000000

Phi=0.511413 Log(Det|R|)=-6.084417 Bartlett Test=304193.45 DF=28 Prob=0.000000

Bar Chart of Absolute Correlation Section

	Variables				
Variables	**codebase_size_loc**	**num_peripherals**	**iteration_count**	**num_minor_changes**	**num_major_changes**
codebase_size_loc		|||||||||||||||	||||||	||||||	|||||||	||||||
num_peripherals	|||||||||||||||		||||||	|||||	|||||||	||||||
iteration_count	||||||	||||||		|||||||||||||||||||	|||	
num_minor_changes	||||||	|||||	|||||||||||||||||		|||	
num_major_changes	|||||||	|||||||	|||	|||		
hw_review_time_hrs	|||||||||	|||||||||	||||||||||||||	||||||||||||||	||||
sw_review_time_hrs	|||||||||	|||||||||	||||||||||||||	||||||||||||||	|||||
verification_pass_rate	||||||||||||||	||||||||||||||	|||||	|||||	||||||||||||

Phi=0.511413 Log(Det|R|)=-6.084417 Bartlett Test=304193.45 DF=28 Prob=0.000000

Factor Analysis Report

Dataset C:\Users\vboxuser\Documents\hal_shift.NCSS

Bar Chart of Absolute Correlation Section

	Variables			
Variables	**hw_review_time_hrs**		**sw_review_time_hrs**	**verification_pass_rate**
codebase_size_loc	|||||||||	||||||||||	||||||||||||	
num_peripherals	||||||||	|||||||||	||||||||||||	
iteration_count	|||||||||||||	|||||||||||||	|||||	
num_minor_changes	|||||||||||||	|||||||||||||	|||||	
num_major_changes	||||	|||||	||||||||||	
hw_review_time_hrs		|||||||||||||	|||||||	
sw_review_time_hrs	|||||||||||||		|||||||||	
verification_pass_rate	||||||||	|||||||||		

Phi=0.511413 Log(Det|R|)=-6.084417 Bartlett Test=304193.45 DF=28 Prob=0.000000

Eigenvalues after Varimax Rotation

No.	Eigenvalue	Individual Percent	Cumulative Percent	Scree Plot
1	3.063863	54.75	54.75	||||||||||
2	2.524600	45.11	99.86	|||||||||
3	0.239634	4.28	104.14	|
4	0.050447	0.90	105.04	|
5	-0.025864	-0.46	104.58	|
6	-0.038196	-0.68	103.90	|
7	-0.081311	-1.45	102.44	|
8	-0.136699	-2.44	100.00	|

Eigenvectors after Varimax Rotation

	Factors	
Variables	**Factor1**	**Factor2**
codebase_size_loc	0.349693	-0.403463
num_peripherals	0.322473	-0.377553
iteration_count	0.385578	0.419892
num_minor_changes	0.386206	0.423480
num_major_changes	0.170301	-0.271796
hw_review_time_hrs	0.400226	0.194123
sw_review_time_hrs	0.420611	0.136494
verification_pass_rate	-0.331366	0.456988

Bar Chart of Absolute Eigenvectors after Varimax Rotation

	Factors	
Variables	**Factor1**	**Factor2**
codebase_size_loc	|||||||	||||||||
num_peripherals	|||||||	|||||||
iteration_count	||||||||	||||||||
num_minor_changes	||||||||	||||||||
num_major_changes	||||	|||||
hw_review_time_hrs	||||||||	||||
sw_review_time_hrs	||||||||	|||
verification_pass_rate	|||||||	||||||||

Factor Analysis Report

Dataset C:\Users\vboxuser\Documents\hal_shift.NCSS

Factor Loadings after Varimax Rotation

	Factors	
Variables	**Factor1**	**Factor2**
codebase_size_loc	0.234576	-0.831136
num_peripherals	0.212043	-0.771759
iteration_count	0.931195	-0.079508
num_minor_changes	0.934970	-0.076827
num_major_changes	0.055670	-0.477650
hw_review_time_hrs	0.778578	-0.316468
sw_review_time_hrs	0.765724	-0.397929
verification_pass_rate	-0.164209	0.859842

Bar Chart of Absolute Factor Loadings after Varimax Rotation

	Factors	
Variables	**Factor1**	**Factor2**
codebase_size_loc	|||||	||||||||||||||
num_peripherals	|||||	||||||||||||||
iteration_count	||||||||||||||||||	||
num_minor_changes	||||||||||||||||||	||
num_major_changes	||	|||||||||
hw_review_time_hrs	|||||||||||||||	|||||||
sw_review_time_hrs	|||||||||||||||	||||||||
verification_pass_rate	||||	|||||||||||||||||

Communalities after Varimax Rotation

	Factors		
Variables	**Factor1**	**Factor2**	**Communality**
codebase_size_loc	0.055026	0.690787	0.745813
num_peripherals	0.044962	0.595612	0.640574
iteration_count	0.867124	0.006322	0.873446
num_minor_changes	0.874169	0.005902	0.880072
num_major_changes	0.003099	0.228149	0.231248
hw_review_time_hrs	0.606184	0.100152	0.706336
sw_review_time_hrs	0.586334	0.158347	0.744681
verification_pass_rate	0.026965	0.739329	0.766294

Bar Chart of Communalities after Varimax Rotation

	Factors		
Variables	**Factor1**	**Factor2**	**Communality**
codebase_size_loc	||	|||||||||||||	|||||||||||||
num_peripherals	|	|||||||||||	|||||||||||
iteration_count	||||||||||||||||||	|	||||||||||||||||
num_minor_changes	||||||||||||||||||	|	||||||||||||||||
num_major_changes	|	|||||	|||||
hw_review_time_hrs	|||||||||||||	|||	|||||||||||||
sw_review_time_hrs	|||||||||||	||||	|||||||||||||
verification_pass_rate	|	|||||||||||||	||||||||||||||

Factor Analysis Report

Dataset C:\Users\vboxuser\Documents\hal_shift.NCSS

Factor Structure Summary after Varimax Rotation

	Factors
Factor1	**Factor2**
num_minor_changes	verification_pass_rate
iteration_count	codebase_size_loc
hw_review_time_hrs	num_peripherals
sw_review_time_hrs	num_major_changes

Plots Section

